SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1708**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

*I P **DEENADAYALAN (192565097**) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
P Deenadayalan

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

1. AI-Powered Drone Route Planning in a Flood-Affected Region

i. Greedy Best-First Search (GBFS) in this Scenario

Given:

- AI-powered drone delivering emergency medicines.
- Dynamic environment with blocked roads and changing weather.
- Uses only heuristic (straight-line distance) to reach the destination quickly.

Explanation:

Greedy Best-First Search selects the path that appears **closest to the goal** based only on the heuristic value **h(n)**.

Formula:

f(n)=h(n)f(n)=h(n)f(n)=h(n)

Strengths:

- ✅ Very fast in finding a route.
- ✅ Explores fewer nodes.
- ✅ Suitable for urgent situations where quick decisions are needed.

Weaknesses:

- ❌ Ignores actual travel cost.
- ❌ May choose blocked or risky routes.
- ❌ Does not guarantee the shortest or safest path.
- ❌ Performance decreases in uncertain and changing environments.

*ii. A Search in this Scenario**

Explanation:

A* Search considers both:

- Actual cost from the start **g(n)**
- Estimated cost to the goal **h(n)**

Formula:

f(n)=g(n)+h(n)f(n)=g(n)+h(n)f(n)=g(n)+h(n)

How it Works:

- Calculates the total estimated cost for every possible path.
- Avoids expensive or dangerous routes.
- Updates its path when movement costs change.
- Finds an optimal route if the heuristic is admissible.

Advantages:

- ✅ Finds the shortest and safest path.
- ✅ Balances speed and travel cost.
- ✅ Better suited for changing terrain and weather.
- ✅ Guarantees optimal solution with a good heuristic.

iii. Impact of Heuristic Accuracy

For Greedy Best-First Search

- **Accurate heuristic:** Quickly reaches the destination.
- **Poor heuristic:** May follow wrong or blocked paths and waste time.

For A*

- **Accurate heuristic:** Faster search with fewer explored nodes.
- **Poor heuristic:** Still finds the optimal path but explores more nodes, increasing computation time.

Analysis

- GBFS depends heavily on heuristic quality.
- A* is more reliable because it also considers actual path cost.

iv. Effect of Dynamic Changes (Blocked Paths)

Greedy Best-First Search

- Frequently gets misled by blocked paths.
- May need repeated searching.
- Search efficiency decreases significantly.

A*

- Recalculates path using updated costs.
- Can avoid blocked or high-cost routes.
- Handles environmental changes more effectively, though with additional computation.

Comparison

Dynamic obstacles reduce the efficiency of both algorithms, but **A*** adapts more effectively because it evaluates both cost and heuristic.

v. Recommended Algorithm

*Recommendation: A Search**

Justification:

Criteria	Greedy Best-First Search	A* Search
Real-time Decision	Very Fast	Fast
Considers Travel Cost	✗ No	✓ Yes
Handles Dynamic Changes	Poor	Good
Finds Optimal Path	✗ No	✓ Yes
Safety	Low	High
Reliability	Moderate	Excellent

Conclusion

*A Search is the most suitable algorithm** for emergency medicine delivery because it balances **actual travel cost** and **heuristic estimation**, enabling the drone to choose the **fastest, safest, and most reliable route**. Although it requires slightly more computation than Greedy Best-First Search, its ability to produce **optimal paths** and adapt to changing conditions makes it the best choice for real-time disaster response.

2. Smart City AI Traffic Signal Optimization

Given:

- AI system controls traffic signals at hundreds of intersections.

- Objectives:
    - Minimize total waiting time.
    - Minimize fuel consumption.
    - Reduce traffic congestion.
- Traffic changes continuously.
- Search space is extremely large.

Formulation as an Optimization Problem

Objective Function

Find the traffic signal timings that minimize:

- Total waiting time
- Fuel consumption
- Traffic congestion

Optimization Goal

Minimize f(x)=Waiting Time+Fuel Consumption+Congestion\text{Minimize } f(x)=\text{Waiting Time}+\text{Fuel Consumption}+\text{Congestion}Minimize f(x)=Waiting Time+Fuel Consumption+Congestion

where **x** represents the signal timings at all intersections.

Why Traditional Search is Inefficient

- Very large number of possible signal combinations.
- Traffic conditions change continuously.
- Exhaustive search requires excessive time and memory.
- Cannot provide real-time solutions.

i. Hill Climbing in this Scenario

Explanation

Hill Climbing starts with an initial traffic signal plan and repeatedly changes the signal timings to improve traffic flow.

Working

1. Select an initial signal timing.
2. Generate neighboring solutions by slightly changing signal timings.
3. Move to the solution with the best improvement.
4. Repeat until no better neighbor exists.

Advantages

- ✅ Simple to implement.
- ✅ Fast.
- ✅ Uses little memory.
- ✅ Suitable for local optimization.

Limitations

- ❌ May stop at a local optimum.
- ❌ Cannot guarantee the best solution.
- ❌ Sensitive to the starting point.
- ❌ Performs poorly in complex search spaces.

ii. Local Maxima, Plateaus, and Ridges

1. Local Maxima

A solution that is better than nearby solutions but not the global best.

Real-world Example:
One intersection has excellent traffic flow, but nearby intersections become heavily congested.

2. Plateaus

A flat region where neighboring solutions have nearly the same quality.

Real-world Example:
Changing signal timing by a few seconds produces no noticeable improvement in traffic.

3. Ridges

The optimal path requires several coordinated changes, but Hill Climbing changes only one step at a time.

Real-world Example:
Improving traffic requires adjusting multiple intersections simultaneously, which Hill Climbing cannot easily achieve.

iii. Simulated Annealing and Genetic Algorithms

A. Simulated Annealing

Working

- Sometimes accepts worse solutions.
- Escapes local maxima.
- Gradually becomes more selective as the temperature decreases.

Advantages

- ✓ Escapes local optima.
- ✓ Handles dynamic optimization.
- ✓ Better than Hill Climbing.

B. Genetic Algorithms

Working

1. Generate multiple traffic signal plans.
2. Evaluate their performance.
3. Select the best plans.
4. Apply crossover and mutation.
5. Produce improved generations.

Advantages

- ✓ Searches many solutions simultaneously.
- ✓ Avoids local optima.
- ✓ Handles very large search spaces.

- ✅ Highly scalable.
- ✅ Adapts to changing traffic conditions.

iv. Recommended Approach

Recommendation: Genetic Algorithm

Justification

Criteria	Hill Climbing	Simulated Annealing	Genetic Algorithm
Large Search Space	Poor	Good	Excellent
Escapes Local Optima	❌ No	✅ Yes	✅ Yes
Scalability	Low	Medium	High
Adaptability	Low	Good	Excellent
Global Optimization	Poor	Better	Excellent
Real-Time Performance	Moderate	Good	Excellent

Conclusion

The **Genetic Algorithm** is the most suitable approach because it explores multiple solutions simultaneously, avoids local optima through crossover and mutation, scales efficiently to hundreds of intersections, and adapts well to continuously changing traffic patterns. It provides the **best balance of scalability, adaptability, and optimization**, making it ideal for smart city traffic signal control.

3. Autonomous Mars Rover – Online Search Agent

Given

- Autonomous rover explores unknown terrain on Mars.
- Objectives:
    - Navigate safely.
    - Collect samples.
    - Avoid hazards (craters and rocks).
    - Operate with limited sensing capability.
    - No complete map is available.
    - Learns about the environment while moving.

i. Why Online Search is Required Instead of Offline Search

Explanation

- **Offline Search** requires complete knowledge of the environment before planning a path.
- Since the Mars rover has **no complete map**, it cannot determine the entire route in advance.
- **Online Search** allows the rover to make decisions while exploring and update its path using newly discovered information.

Reasons

- Unknown environment.
- Limited sensing capability.
- Hazards are discovered during movement.
- Real-time decision-making is required.

Answer

Therefore, an **Online Search Agent** is more suitable because it can continuously explore, learn, and safely navigate without prior knowledge of the environment.

ii. Challenges of Partial Observability and Dynamic Environments

Partial Observability

The rover can observe only the nearby surroundings and cannot view the entire terrain.

Challenges

- Hidden obstacles such as rocks and craters.
- Unknown paths.
- Incomplete environmental information.
- Increased risk of unsafe navigation.

Dynamic Environment

The environment may change while the rover is moving.

Examples

- Dust storms reduce visibility.
- Loose rocks may shift position.
- New hazards may appear unexpectedly.

Impact

- Previously planned paths may become unsafe.
- Continuous replanning is necessary.

- Computational complexity increases.

Answer

The rover must continuously sense its surroundings and adapt to environmental changes to ensure safe and efficient navigation.

iii. How the Rover Builds and Updates its Internal Model

Working

1. Sense the nearby environment using onboard sensors.
2. Detect rocks, craters, and safe paths.
3. Store newly discovered information in memory.
4. Update the internal map after every movement.
5. Recalculate the next action using the updated map.

Benefits

- Improves navigation accuracy.
- Avoids repeated exploration.
- Learns the environment over time.
- Makes safer and more efficient decisions.

Answer

The rover continuously updates its internal model based on sensor data, allowing it to make better navigation decisions as it explores.

iv. Comparison of Search Strategies

Feature	Uninformed Search	Informed Search	Online Search
Requires Complete Map	Yes	Yes	No
Uses Heuristic	No	Yes	May use heuristics
Handles Unknown Environment	Poor	Limited	Excellent
Real-Time Decision Making	No	Limited	Yes
Adaptability	Low	Medium	High
Suitable for Mars Rover	No	Limited	Best

Answer

Online Search is superior because it can operate in unknown environments, adapt to changes, and make decisions in real time.

v. Recommended Approach

Recommendation: Online Search Agent

Justification

Criteria	Online Search Agent
Handles Uncertainty	Excellent
Works Without Complete Map	Yes
Learns the Environment	Yes
Adapts to New Hazards	Yes
Real-Time Decision Making	Excellent
Safety	High

Conclusion

The **Online Search Agent** is the most suitable approach because it makes decisions while exploring, continuously updates its internal map, adapts to newly discovered hazards, and safely navigates unknown terrain. It provides **high adaptability, real-time decision-making, continuous learning, and safe navigation**, making it ideal for autonomous Mars rover exploration.

4. University Exam Timetable Generation – Constraint Satisfaction Problem (CSP)

Given

The university must generate an exam timetable while satisfying the following constraints:

- No student should have overlapping exams.
- Limited number of exam halls.
- Certain subjects must be scheduled before others.
- Faculty availability constraints.
- Special accommodations for some students.
- Number of courses and students is very large.

i. Formulate the Problem as a CSP

A **Constraint Satisfaction Problem (CSP)** consists of:

- **Variables**
- **Domains**
- **Constraints**

The objective is to assign a valid exam time and hall to every course while satisfying all constraints.

ii. Variables, Domains, and Constraints

Variables

Each course is treated as a variable.

Example:

- C1 = Mathematics
- C2 = Physics
- C3 = Chemistry
- C4 = Computer Science

Domains

The possible values assigned to each variable.

Example:

- Time Slots = {Morning, Afternoon, Evening}
- Exam Halls = {Hall A, Hall B, Hall C}

Each course must be assigned one time slot and one hall.

Constraints

1. Student Constraint

A student cannot have two exams at the same time.

2. Hall Constraint

The number of exams in a time slot must not exceed the available exam halls.

3. Subject Order Constraint

Some subjects must be conducted before others.

Example:
Programming → Data Structures

4. Faculty Availability Constraint

Faculty members must be available during their assigned exam time.

5. Special Accommodation Constraint

Students requiring special facilities must be assigned suitable halls.

iii. Backtracking Search

Working

1. Assign a time slot to the first course.
2. Check whether all constraints are satisfied.
3. If valid, assign the next course.
4. If a constraint is violated, return (backtrack) and try another assignment.
5. Continue until all courses are scheduled.

Advantages

- ✅ Guarantees a valid timetable if one exists.
- ✅ Systematically explores solutions.

Limitation

- ❌ Slow for large universities with many courses.

iv. Constraint Propagation (Forward Checking)

Explanation

Forward Checking removes invalid choices before they are selected.

Working

- After assigning a course, eliminate conflicting time slots from related courses.
- Detect impossible schedules early.
- Reduce unnecessary backtracking.

Advantages

- ✅ Faster search.
- ✅ Fewer invalid assignments.
- ✅ Reduces computation time.
- ✅ Improves scalability.

v. Real-World Challenges and Best Strategy

Challenges

- Thousands of students and courses.
- Limited exam halls.
- Faculty scheduling conflicts.
- Last-minute timetable changes.
- Multiple constraints increase complexity.

Best Strategy

Use **Backtracking Search with Forward Checking (Constraint Propagation)**.

Justification

Criteria	Backtracking + Forward Checking
Handles Constraints	Excellent
Reduces Search Space	Yes
Scalability	High
Efficiency	High
Produces Valid Timetable	Yes

Conclusion

The exam scheduling problem is naturally modeled as a **Constraint Satisfaction Problem (CSP)**. **Backtracking Search** systematically assigns exam slots, while **Forward Checking** eliminates invalid choices early, greatly improving efficiency. This combination provides a **scalable, efficient, and reliable solution** for generating university exam timetables.

5. Strategic Game AI – Minimax & Alpha-Beta Pruning

Given

The AI opponent must:

- Compete against human players.
- Make decisions within strict time limits.
- Evaluate many possible moves.
- Handle a very large decision tree.

i. How Minimax Models the Problem

Explanation

The **Minimax algorithm** models the game as a two-player competitive search.

- **MAX player** → AI tries to maximize its score.
- **MIN player** → Opponent tries to minimize the AI's score.

The AI explores future game states and chooses the move with the best worst-case outcome.

ii. Computational Challenges of Large Game Trees

Challenges

- Huge branching factor.
- Millions of possible game states.

- High memory usage.
- Long computation time.
- Difficult to meet real-time deadlines.

Impact

Searching the complete game tree is computationally expensive.

iii. Alpha-Beta Pruning

Explanation

Alpha-Beta Pruning improves Minimax by eliminating branches that cannot influence the final decision.

Working

- **Alpha (α):** Best value found for the MAX player.
- **Beta (β):** Best value found for the MIN player.
- If **α ≥ β**, the remaining branch is ignored (pruned).

Advantages

- ✓ Reduces the number of evaluated nodes.
- ✓ Produces the same optimal result as Minimax.
- ✓ Speeds up decision making.
- ✓ Ideal for real-time games.

iv. Evaluation Function

The evaluation function estimates the quality of a game state.

Example

Evaluation Score=(Resources)+(Army Strength)+(Map Control)+(Health)−(Enemy Advantage)\text{Evaluation Score} = (\text{Resources}) + (\text{Army Strength}) + (\text{Map Control}) + (\text{Health}) - (\text{Enemy Advantage})Evaluation Score=(Resources)+(Army Strength)+(Map Control)+(Health)−(Enemy Advantage)

Factors Considered

- Available resources.
- Number and strength of units.
- Control of strategic locations.
- Player health.
- Enemy strength and threats.

Justification

These factors help the AI estimate which game state is more likely to lead to victory.

v. Recommended Strategy

Recommendation

Use **Minimax with Alpha-Beta Pruning** and a strong evaluation function.

Justification

Criteria	Minimax	Alpha-Beta Pruning
Optimal Decision	Yes	Yes
Search Speed	Slow	Fast
Real-Time Performance	Poor	Excellent
Memory Usage	High	Lower
Scalability	Limited	High

Conclusion

Minimax with Alpha-Beta Pruning is the best approach because it maintains optimal decision quality while reducing the number of evaluated game states. Combined with a well-designed evaluation function, it enables the AI to make **fast, intelligent, and effective decisions** within real-time constraints.